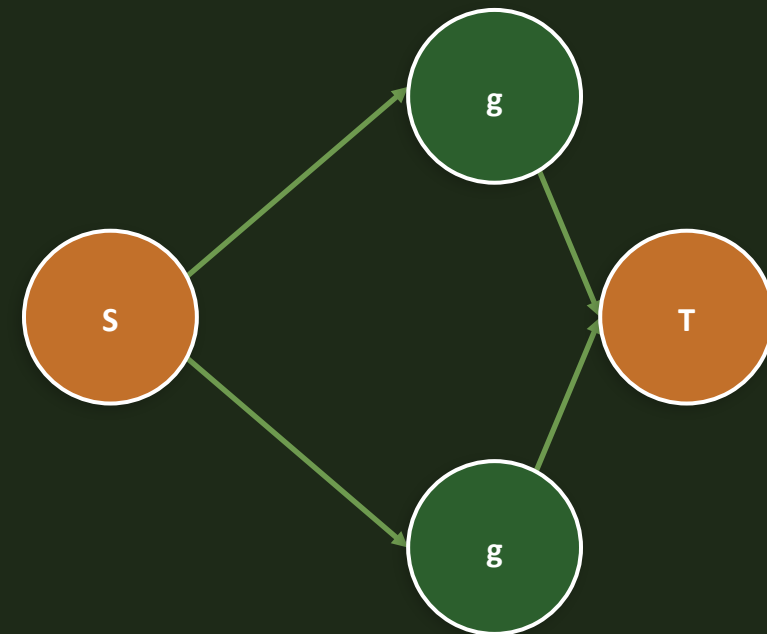


# Gopher II

UVA 10080 — Emparelhamento bipartido modelado como rede de fluxo

---

**Grupo F** · (integrantes: preencher nomes e RAs)



# Contexto e objetivo

**n gophers e m buracos** em coordenadas  $(x, y)$  no plano. Um falcão chega.

Um gopher se salva se alcança um buraco em  $\leq s$  **segundos** à velocidade **v**.

**Cada buraco abriga no máximo 1 gopher** (recurso disputado).

*É um problema de emparelhamento bipartido gophers  $\leftrightarrow$  buracos.*

## OBJETIVO

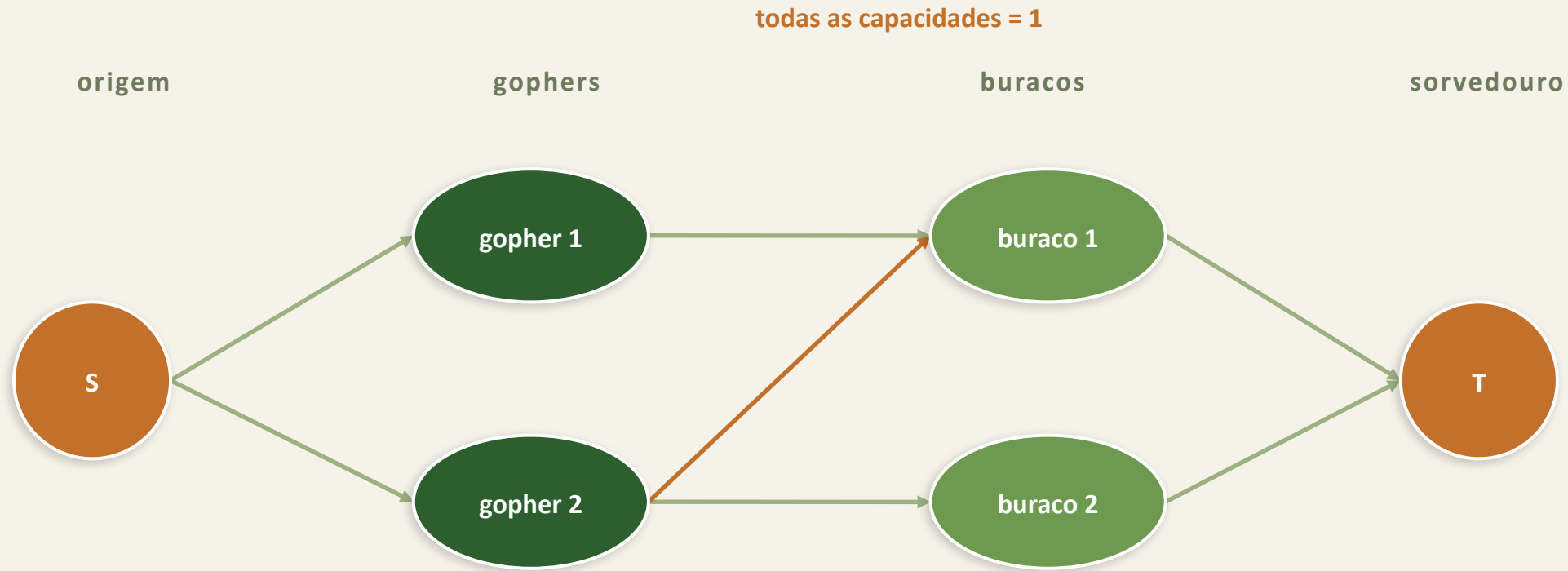
Minimizar gophers vulneráveis  
= **MAXIMIZAR gophers salvos**

$$\text{alcança} \Leftrightarrow \text{distância} \leq s \cdot v$$

$$\Leftrightarrow \text{distância}^2 \leq (s \cdot v)^2$$

*Comparar quadrados evita raiz quadrada e erro de ponto flutuante.*

# Modelagem: a rede de fluxo



**1 unidade de fluxo = 1 gopher que ocupa 1 buraco** (uma decisão válida do enunciado)

# Vértices, arestas e capacidades

| Componente         | Papel no contexto do problema                               | Cap. |
|--------------------|-------------------------------------------------------------|------|
| S — origem         | empurrar 1 unidade por gopher = "tentar salvar este gopher" | —    |
| gophers (camada 1) | um vértice por gopher                                       | —    |
| buracos (camada 2) | um vértice por buraco                                       | —    |
| T — sorvedouro     | chegar em T = gopher escondido (salvo)                      | —    |
| S → gopher         | cada gopher se salva no máximo uma vez                      | 1    |
| gopher → buraco    | criada só se o gopher alcança o buraco — é uma fuga válida  | 1    |
| buraco → T         | cada buraco abriga no máximo um gopher                      | 1    |

## Por que cap. = 1?

**Unitária**  
recurso indivisível, usado uma única vez (1 gopher / 1 buraco).

**Seria  $\infty$**   
se não houvesse limite de passagem na aresta.

**Seria = k**  
se um recurso tivesse k cópias (não é o caso aqui).

*Sem arestas paralelas, não-direcionadas ou múltiplas cópias neste problema.*

valor do fluxo = nº de pares = RESPOSTA

# Estratégia: qual algoritmo de fluxo máximo?

## Ford-Fulkerson (DFS)

- caminho aumentante por busca simples (DFS)
- ok em redes pequenas / capacidades unitárias
- DFS pode escolher caminhos ruins

## Edmonds-Karp (BFS) ✓ escolhido

- caminho aumentante sempre pela BFS (mais curto em nº de arestas)
- comportamento previsível
- evita os caminhos ruins da DFS pura

### Por que Edmonds-Karp?

**Capacidades unitárias**  $\Rightarrow$  cada caminho aumentante soma exatamente 1 ao fluxo, e o fluxo máximo é  $\leq \min(n, m) < 100$ . A BFS é previsível e folgada para  $n, m < 100$ ; é a opção recomendada em caso de dúvida.

# Grafo residual e caminhos aumentantes

- 1 **Capacidade residual** = capacidade – fluxo. Quanto ainda cabe na aresta.
- 2 **Aresta reversa (cap 0)** cada aresta tem sua reversa; a reversa de  $e_{id}$  é  $e_{id}^{-1}$ .
- 3 **Por que reversa?** permite DESFAZER uma escolha: remaneja um gopher para liberar um buraco disputado.
- 4 **Caminho aumentante** BFS encontra um caminho  $S \rightarrow T$  só por arestas com residual  $> 0$ .
- 5 **Gargalo** menor capacidade residual ao longo do caminho (aqui sempre 1).
- 6 **Atualização** +gargalo na aresta direta, –gargalo na reversa.
- 7 **Parada** quando a BFS não acha mais nenhum caminho  $S \rightarrow T$  no residual.

**Por que é ótimo?** Teorema fluxo-máximo / corte-mínimo: sem caminho aumentante  $\Rightarrow$  fluxo máximo  $\Rightarrow$  emparelhamento máximo.

# Do fluxo à resposta

$$\text{vulneráveis} = n - \text{fluxo máximo}$$

Salvos = fluxo máximo = emparelhamento máximo.

**Reconstrução do pareamento:** as arestas gopher → buraco com  $\text{fluxo} > 0$  dizem qual gopher foi para qual buraco.

- *O enunciado pede só a quantidade ⇒ basta o valor do fluxo.*

Exemplo do enunciado ·  $\text{alcance}^2 = (5 \cdot 10)^2 = 2500$

|                | buraco 1 (100,100) | buraco 2 (20,20) |
|----------------|--------------------|------------------|
| gopher 1 (1,1) | 19602 ✗            | 722 ✓            |
| gopher 2 (2,2) | 19208 ✗            | 648 ✓            |

*Só  $g1 \rightarrow b2$  e  $g2 \rightarrow b2$  são alcançáveis; buraco 1 é inalcançável e b2 só salva um.*

$$\begin{aligned} \text{fluxo máximo} &= 1 \\ \text{vulneráveis} &= 2 - 1 = 1 \quad \checkmark \text{ (Sample Output)} \end{aligned}$$

# Complexidade e casos especiais

## Complexidade

Edmonds-Karp:  $O(V \cdot E^2)$  no pior caso.

$V = n + m + 2$  e  $E = O(n \cdot m)$ , com  $n, m < 100$ .

**Capacidades unitárias**  $\Rightarrow \leq \min(n, m)$  aumentos  $\times$  BFS  $O(V+E)$ .

**Memória:** dominada pela lista de arestas residuais,  $O(n \cdot m)$ .

*Cada caso de teste roda praticamente instantâneo.*

## Casos especiais

- Vários casos de teste até EOF.
- Recursos insuficientes / buracos inalcançáveis  $\Rightarrow$  sobra vulnerável (coberto por  $n - \text{fluxo}$ ).
- Comparação por distância<sup>2</sup> evita raiz e erro de ponto flutuante.
- Nenhum gopher alcança buraco  $\Rightarrow$  fluxo 0  $\Rightarrow$  todos vulneráveis.
- INF seguro (`float('inf')`) usado só no cálculo do gargalo.
- BOM no início da entrada é ignorado por segurança.



# Conclusão

---

“Esconder gophers” reduz a **fluxo máximo bipartido com capacidades 1** — solução ótima.

A resposta é direta: **vulneráveis** =  $n - \text{fluxo máximo}$ .

**Edmonds-Karp com grafo residual e arestas reversas, implementado do zero** (sem bibliotecas de fluxo).

Submissão: Accepted na UVA

Grupo F · Obrigado!